

Operational Test: Cell-Behavior-Determinism Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone operational procedure, the test that verifies the **cell-behavior-determinism guarantee** within the CKS determinism contract — the guarantee that cells produce deterministic substrate state changes given substrate state, orchestration rule, and recorded bounded non-determinism — so that downstream implementers and auditors can refer to the test as a single citable specification of what their deployments must verify on the cell-behavior axis.

Abstract

The CKS determinism contract names five guarantees that any contract-coherent substrate must satisfy. A separate note formalizes that contract as a unit; subsequent decomposition notes formalize each guarantee individually. One of them — *cell-behavior determinism* — commits cells to producing equivalent substrate writes when executing under the same orchestration rule over the same substrate state, modulo bounded non-determinism arising from LLM consultations and other recorded non-deterministic inputs. This note formalizes the test that verifies that guarantee specifically. The test handles bounded non-determinism through a replay procedure: record the LLM outputs and other bounded non-deterministic inputs in a first execution, replay them in a second execution, and verify that the substrate state change is identical across the replayed pair. The note states what the test verifies precisely, specifies the procedure as operational steps, names the pass and fail criteria, enumerates the anti-patterns the test detects, describes the deployment-verification points at which the test should run, and specifies the architectural limits of the test.

1. Why the cell-behavior-determinism test needs to be formalized as standalone

The CKS pattern's determinism contract distributes its commitments across §3.3, §4.1, §4.4, §6.2, and §11.3 of the source paper. One of the contract's five guarantees commits cells to producing equivalent substrate writes under the same orchestration rule over the same substrate state, modulo recorded bounded non-determinism. The guarantee is the foundation of three further architectural properties: reproducibility of audit replay, interpretability of cell behavior under change, and the substrate's status as the deterministic side

of the governance boundary. If cell-behavior determinism cannot be verified, none of the three follows.

The test that verifies it has a specific shape that distinguishes it from naive determinism testing applied to AI-mediated systems. Naive testing requires the same input to produce the same output across executions; this is not what the contract requires of cells, because the LLM consultations cells perform are inherently non-deterministic. The contract's actual commitment is *replay determinism*: given the same substrate state, the same orchestration rule, and the same recorded LLM outputs (and other bounded non-deterministic inputs), the cell produces the same substrate writes. Getting the framing wrong in either direction — toward stricter execution determinism or toward looser non-replay tolerance — produces a test that does not verify what the contract actually commits to.

Standalone formalization is what makes the test independently citable, runnable, and scopable. Deployments verify cell-behavior determinism at distinct moments — at initial activation, after cell-architecture changes, after library-version updates, after LLM-vendor changes, and as a composition-partner check — and at each, the test runs by itself, against the cell-behavior axis specifically, without entangling other guarantees the broader contract makes.

2. The commitment under test

The test verifies one specific commitment from the determinism contract: cells executing under the same orchestration rule over the same substrate state produce equivalent substrate writes, with equivalence judged at the substrate-write layer (what state changes are committed) rather than at the LLM-output layer (what intermediate model output the cell produced en route).

The commitment has three components.

Substrate-state determinism of writes. Given identical substrate state at the start of execution, identical orchestration-rule input, and identical recorded bounded non-deterministic inputs, the cell must produce identical substrate writes. Identity is judged at the substrate-write layer — same writes, same target locations, same provenance fields, same conflict-resolution outcomes — not at the level of the LLM's intermediate text.

Bounded non-determinism is recorded, not eliminated. The contract does not require LLM consultations to become deterministic; it requires that any non-deterministic input the cell consumes is recorded as part of the cell's provenance, so that the execution can be replayed deterministically. The recorded categories include LLM outputs, deliberately invoked randomness, wall-clock-time references, and similar inputs whose non-determinism is bounded by being recorded.

Cell-internal state does not persist across executions in a way that affects substrate writes. A cell's behavior is a function of substrate state, orchestration rule, and recorded inputs only. Cell-internal state — caches, accumulators, session-scoped storage — that persists across executions and changes substrate writes is outside the function's specification, and any cell whose behavior depends on such state fails the commitment.

The test verifies all three by running the same cell over the same substrate state with the same recorded inputs twice and comparing the substrate writes.

3. The replay procedure: handling bounded non-determinism

The test's central mechanism is a replay procedure that converts bounded non-determinism into a property the test can hold constant across paired executions.

In the first execution of a paired test run, the cell is executed against a known substrate state under a chosen orchestration rule. As the cell runs, every bounded non-deterministic input it consumes is recorded as part of the cell's provenance: LLM consultation requests and the responses received, any randomness invoked under the orchestration rule's authority, any wall-clock-time references the cell uses, and any other inputs whose non-determinism the contract bounds by recording. The substrate-write set produced by the execution is also recorded.

In the second execution, the substrate is reset to the same starting state. The cell is run again with the same orchestration rule, but the recorded bounded non-deterministic inputs from the first execution are replayed in place of fresh consultations or invocations. Each LLM consultation receives the recorded response; each randomness invocation receives the recorded value; each wall-clock-time reference receives the recorded timestamp. The cell sees a world identical to the one it saw in the first execution, including the inputs that would otherwise have been non-deterministic. The substrate-write set from the second execution is then compared to the recorded substrate writes from the first.

The replay procedure is what distinguishes the cell-behavior-determinism test from execution-determinism tests applied to deterministic procedures. The test does not require the cell to make the LLM deterministic. It requires the cell to be a deterministic function of inputs that include the LLM's recorded outputs.

4. The test procedure and pass/fail criteria

The test runs against a single cell, a single orchestration rule, and a chosen starting substrate state.

1. **Set up.** Establish substrate state X and orchestration rule R .
2. **First execution with recording.** Execute the cell over X under R . Record every bounded non-deterministic input the cell consumes. Observe the substrate transition $X \rightarrow X'$ and record the substrate-write set that produced it.
3. **Reset.** Restore the substrate to state X . Any side-effect state the cell may have written to substrate-adjacent stores must also be cleared.
4. **Second execution with replay.** Execute the cell again over X under R , replaying the recorded inputs from Step 2. Observe the substrate transition $X \rightarrow X''$ and record the substrate-write set that produced it.
5. **Compare.** Compare the substrate-write sets and the resulting substrate states.

The test **passes** when X' equals X'' and the substrate-write sets are identical at the substrate-write layer (same writes, same locations, same provenance fields), the bounded non-deterministic inputs were recorded in the first execution and replayed in the second without divergence, and the second execution did not consume any non-deterministic input that was not recorded.

The test **fails** under three failure modes. **Divergent writes:** X' differs from X'' , or the substrate-write sets differ; the cell consumed input outside its specified function. **Recording incompleteness:** the second

execution attempts to consume a non-deterministic input for which no recorded value exists. The cell reads a non-deterministic source the provenance recording does not cover; recording incompleteness is itself a contract violation, because bounded non-determinism that is not recorded is not bounded. **Replay collision:** the second execution consumes the recorded inputs in a different order, at a different point, or under different conditions than the first, such that replay cannot be matched to consumption. This indicates that the cell's input-consumption pattern is itself non-deterministic, a deeper specification error than recording incompleteness.

In all three failure modes, the cell's behavior is non-deterministic in the contract's sense, and the failure must be diagnosed and corrected before the cell can be activated against the deployed substrate.

5. Anti-patterns the test detects

Each of the following ways a cell's behavior can depend on input outside the function's specification produces test failure under one of the three failure categories.

Implicit context in cells. The cell reads context that is neither part of substrate state nor part of recorded bounded non-deterministic inputs — configuration values not represented in the substrate, organizational state held in adjacent systems but not addressable from the substrate, derived state computed by an upstream cell but not written back. The test fails because the implicit-context source's state may differ between executions.

Hidden cell state. The cell holds state across executions — caches, accumulators, internal counters, session-scoped memory — that affects its substrate writes. The second execution sees the state populated by the first; divergent writes follow.

LLM-as-terminal-producer. The LLM's output flows directly into a substrate write without rule-mediated transformation that anchors the write deterministically to substrate state and recorded inputs. A cell that writes "whatever the LLM said" is not a deterministic function of recorded inputs at the contract layer; the orchestration rule must specify a deterministic transformation of LLM output into substrate write for the cell to satisfy the commitment.

Time-dependent behavior. The cell uses wall-clock time without recording it. The second execution occurs at a different wall-clock time, and writes diverge. The remedy is to record the timestamp as a bounded input.

Environment-dependent behavior. The cell reads environment variables, OS state, the filesystem, network state, or other deployment-environment state directly. The deployment environment is not part of the cell's specified function and is not recorded; the test fails when the environment changes.

Randomness without recording. The cell invokes a randomness source — a random number generator, a UUID generator, a sampling procedure — without recording the values it consumes. The second execution invokes the same source and receives different values.

Library-version-dependent behavior. The cell's substrate writes vary with the version of a library it depends on, without that version being recorded as a bounded input. The dependence is a latent failure that the test surfaces when library updates trigger re-execution against a recorded baseline.

The test detects each as a divergence between the first execution's substrate writes and the second's, or as a recording-incompleteness or replay-collision failure. The test does not name which anti-pattern caused a given failure; diagnosing the cause is a separate engineering activity. What the test guarantees is that the cell, as deployed, is or is not a deterministic function of substrate state, orchestration rule, and recorded bounded inputs.

6. Integration with deployment verification

The test runs at five distinct moments in a deployment's lifecycle.

Initial deployment validation. Before the cell is activated against the deployed substrate, the test is run for each (cell, rule, substrate state) triple that exercises the cell's behavior across cases of interest. A cell that fails initial validation is not activated.

Cell-architecture-change verification. When the cell's logic, orchestration rule, or input-handling pattern changes, the test is re-run. A change that introduces non-determinism — even when it appears unrelated to the cell's substrate-write logic — must fail the test before deployment proceeds.

Library-version-change verification. When a library the cell depends on changes version, the test is re-run against the baseline recorded in the prior version's deployment. A library change that produces divergent substrate writes either requires the cell to be updated to remain deterministic, or requires the library version to be recorded as a bounded input under the contract's allowed-non-determinism categories.

LLM-vendor-update verification. When the LLM vendor, model version, or sampling configuration changes, the test is re-run. The replay procedure protects against the LLM's own non-determinism in normal operation, but a vendor update that changes the cell's input-consumption pattern surfaces as a recording-incompleteness or replay-collision failure.

Composition-partner verification. When the cell is composed with another substrate or another cell, the test is re-run on the composition partner side, to verify that the cell's determinism is preserved across the composition boundary. A composition that breaks cell-behavior determinism breaks the contract for downstream consumers regardless of how the partner systems individually behave.

These five moments are the architecturally load-bearing ones — the points at which a deployment's contract coherence is at stake and the test's pass/fail outcome determines whether deployment proceeds.

7. Limits of the test

The test verifies cell-behavior determinism specifically. It does not verify the contract's other guarantees, and it does not verify properties of the cell that the contract does not address.

It does not verify read determinism. The contract's read-determinism guarantee — that two reads of identical substrate state yield identical content — is verified by a separate test, formalized in the next note in this Phase A5 series.

It does not verify change addressability in full. The substrate-write-set comparison this test performs detects divergent or unaddressable writes, but the contract's full addressability guarantee is verified by a

separate provenance-completeness test elsewhere in this series.

It does not verify conflict-handling determinism. The contract's conflict-handling guarantee — that contradictions are not collapsed by non-deterministic processes — is verified by a separate conflict-handling-determinism test elsewhere in this series.

It does not verify the mediator role. The architectural commitment that the LLM operates as a substrate mediator (rather than as a terminal producer or as a substrate replacement) is verified by the immediately prior test in this series.

It does not verify cell logic correctness. The test verifies that the cell's substrate writes are a deterministic function of substrate state, orchestration rule, and recorded bounded inputs. It does not verify that those writes are *correct* — that they reflect the cell's intended behavior, satisfy domain requirements, or produce desirable outcomes. A cell that produces deterministically wrong writes passes as fully as one that produces deterministically right ones; correctness is a separate concern verified by separate means.

These limits are deliberate. The standalone scoping is what makes the test runnable and citable independently. A test that verified more would entangle concerns the contract decomposes into separate guarantees.

8. Operational test

A cell satisfies the cell-behavior-determinism guarantee for a chosen (cell, rule, substrate state) triple if and only if all of the following are true:

1. The first execution of the cell over substrate state X under rule R produces a substrate transition $X \rightarrow X'$ and records every bounded non-deterministic input consumed during execution.
2. A second execution of the same cell over the same substrate state X under the same rule R , replaying the recorded bounded non-deterministic inputs from the first execution, produces a substrate transition $X \rightarrow X''$.
3. X' equals X'' , and the substrate-write sets produced by the two executions are identical at the substrate-write layer.
4. The second execution consumes only non-deterministic inputs that were recorded in the first; no unrecorded source is read.
5. The second execution consumes recorded inputs in the same order, at the same points, and under the same conditions as the first.

A cell that fails any of (1)–(5) for any (cell, rule, substrate state) triple does not satisfy the cell-behavior-determinism guarantee for that triple, and the failure must be diagnosed and corrected before the cell is deployed against the substrate.

9. Why naming the test as standalone matters

The cell-behavior-determinism test is one of approximately sixteen standalone operational tests in the Phase A5 verification surface. Each formalizes a single test against a single architectural commitment, and each is

independently citable, runnable, and scopable. The standalone framing is what allows deployments to verify each guarantee in isolation, to localize failures precisely, and to extend verification incrementally as new commitments are formalized.

Within Phase A5, this note continues a cluster of tests centered on the AI-mediation-and-substrate-state axis. The immediately prior note formalized the test that verifies the AI-as-substrate-mediator role. The immediately next note will formalize the test that verifies read determinism — the contract's complementary guarantee on the substrate-read side. Subsequent notes in the cluster will cover provenance completeness, accountability-question coverage, conflict-handling determinism, and the remaining contract guarantees as standalone tests.

Naming the test as standalone gives downstream implementers a precise specification of what they must verify on the cell-behavior axis, gives auditors a single citable target, and gives composition partners a clean axis along which to check that determinism is preserved across composition boundaries. It also makes the contract's most operationally subtle commitment — that determinism is *replay determinism*, with bounded non-determinism handled by recording-and-replay rather than by elimination — testable in practice rather than only in principle.

Subsequent work that implements, extends, or argues against the CKS cell-behavior-determinism commitment should use the test in the form formalized here. Subsequent work that verifies the commitment differently is verifying something different from what this note specifies, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Operational Test: Cell-Behavior-Determinism Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.